

Time Partitioning

1 Problem statement

Paxos's main bottleneck is the leader. Decoupling, via Michael's methods, reduces the Paxos leader to a sequencer, broadcaster, and quorum collector. The broadcaster and quorum collection can be trivially partitioned; the sequencer cannot be partitioned and must process every message. In Michael's paper, because Paxos also includes replicas, which processes the committed log in sequence, the replicas end up becoming the bottleneck. This is kind of a distraction: the most scalable variants of Paxos treat it as a log replication (rather than state machine replication) protocol and don't include replicas; in that case, the sequencer becomes the new bottleneck.

Ultimately, removing the bottleneck from Paxos requires partitioning the sequencer. Time partitioning achieves this.

2 Intuition

2.1 Time partitioning for Paxos

Assuming the Paxos leader remains stable (the max ballot does not change often), all the sequencer does is assign each incoming payload a monotonically increasing sequence number. The actual sequence number assigned is arbitrary, based on the order of message delivery. Theoretically, the sequencer could therefore be partitioned so each partition is in charge of a hash subset of sequence numbers: given N partitions, partition 0 will hand out any numbers $x \equiv 0 \pmod N$, partition 1 will hand out any numbers $x \equiv 1 \pmod N$, and so on.

2.2 Time partitioning for any protocol

Although we discuss this partitioning in terms of sequence numbers in Paxos, that is incompatible with how partitioning is implemented. The implementation of partitioning requires selecting a particular attribute of each relation to determine the destination partition; however, when inputs arrive at the sequencer, they do not yet have a sequence number, so it is unclear where the input should be sent. Furthermore, this partitioning is useful even if there is no obvious attribute (such a sequence number) to partition on; consider the case where the

sequence number, instead of being incremented, is summed with the previous sequence number.

What is being partitioned is *time*; at each logical time, if the sequencer receives exactly 1 client input message, then the sequence number will advance by 1. Each of the N partitions then calculates the next sequence number by anticipating N additional inputs that then arrive, one per logical tick, and executing those ticks (and producing no outputs), then processing the next input N ticks later.

Assuming the client will send messages forever, client messages can be round-robin (or randomly) partitioned across the partitions, but if the client will only send a finite number of messages, then the partitions will need to anticipate future ticks that never occur.

This is solved with **AtLeastOnce** streams in Hydro, which allow network message duplication. Partitions can hold onto the first input that they receive and periodically gossip the largest logical tick they have processed. Partitions that have fallen behind significantly due to a drought of inputs can be fed artificially duplicated messages.

The last issue that must be addressed is causality. A client that sends a message, which is processed at logical time t , will expect time to increase monotonically, such that its next message is processed at some t' where $t' > t$. We can address this by attaching a logical clock to each message, and buffering inputs at the partitions until the logical clocks are sufficiently high, similar to the mechanism for partial partitioning.

3 Mechanism

Similar to partial partitioning, each partition maintains a logical clock C_p , forming a vector clock for the entire system. These logical clocks are incremented by partitions.

If the partitions are also partially partitioned, the coordinator separately tracks a logical clock C_c that is incremented whenever the replicated value is changed, forming a lexicographically ordered clock $\langle C_c, C_p \rangle$.

3.1 Time partitioning

Time partitioning is possible if the state of the system at time $t+1$ is a function of the state of the system at time t , regardless of input values.

Slightly more formally, let the **sequential facts** of a relation r at time t be all facts that can be derived by executing a sequential rule at time $t-1$. Let any pair of histories H_1, H_2 be **input cardinality equal** if for all times t , H_1 and H_2 have the same number of facts in each input relation. A relation r is **input-sequential independent** if for all pairs of input cardinality equal histories, for all times, r has the same sequential facts in both histories. Time partitioning is possible for component C if all relations in C are input-sequential independent.

1. All machines maintain a vector clock, which includes a row for this partitioned component.
2. All machines attach the vector clock to all messages they send.
3. Upon receiving a message with a vector clock, non-partitioned machines update their own vector clock by taking the elementwise maximum of their own clock and the received clock.
4. Modify machines that send to the partitioned component:
 - (a) Randomly select a partition to send the message to. (Best for liveness given an unknown number of other senders)
5. Partition the component:
 - (a) Let C_p be the row in the vector clock that represents the partition's logical clock.
 - (b) After processing an input, each partition increments its own C_p by N , where N is the number of partitions.
 - (c) When a partition receives a message with C_p higher than its own value, it buffers the message until its own C_p is sufficiently high.
 - (d) Upon receiving the 1st input, keep it in memory.
 - (e) Periodically gossip the largest C_p that has been processed to all other partitions. The 1st gossip should include its stored input.
 - (f) If its own C_p is significantly behind the largest C_p that has been processed, catch up by processing its stored input until its own C_p is sufficiently high.

3.2 Partial time partitioning

Partial time partitioning is necessary if the state of the system at time $t + 1$ may depend on input values that have arrived at time t . We will split inputs into **replicated** and **partitioned** inputs, where relations r in component C are input-sequential independent on the partitioned inputs.

There is a more fine-grained definition (replicated inputs may only break sequential independence with certain input values) but we will keep things simple for now. In Paxos, inputs that change the ballot number are replicated, while client inputs can be time-partitioned.

The mechanism is a combination of that of partial partitioning and time partitioning.

4 Proof Sketch

Prove that whatever property we use to find input-sequential independent relations is safe. Prove that the state of input-sequential independent relations

at time t is a function of the state of the system at time $t - 1$, regardless of input values. The remainder of the proof is the same as the proof for partial partitioning.

5 Linearizable time partitioning

The partitioning described above is not linearizable, because we need to shift times (the we do in non-blocking partial partitioning) such that the $<$ relation may no longer hold. This presents pretty severe downsides; any communication between any clients must now include our vector clock, making composition difficult if there are multiple optimized protocols with multiple vector clocks. We also claimed in our SIGMOD paper that linearizability is *the gold standard* for correctness, so we should strive to maintain it.

Intuitively, achieving linearizability requires restricting partitions such that they do not execute any ticks until it can guarantee that no other partition will execute an earlier tick using a newer message (that may happen-after this partition's output).

5.1 Mencius-style time partitioning

At the most basic level, Mencius allows the proposer to partition slots across multiple leaders. This means that a message might arrive at a leader responsible for a lower slot and result in an execution impossible in regular MultiPaxos (if the leader does not change). In order to implement *linearizable* Mencius-style time partitioning, coordination between partitions is necessary.

I'll outline potential approaches below. Performance implications require experimentation.

5.1.1 Pipelined coordination

Intuitively, partition 2 should not execute tick 2 until partition 1 tells it that it has executed tick 1. Each partition is modified to send a message to the next partition whenever it executes a tick; the partition is otherwise frozen.

Naively implemented, this is worse than not partitioning, equivalent to executing on a single machine and adding $1/2$ RTT message latency between each input message (and therefore each tick).

There are a couple of ways we can potentially optimize this:

- Dynamically partition ticks so each partition can execute over all messages in its input queue before telling the next partition. Now the latency is not per tick/input message, but per batch.
- Send the tick execution message to the next partition before actually executing that tick; the execution of that tick (or series of ticks, if combined with the optimization above) can then be parallelized. If timed perfectly (the partition finishes executing its batch right as it receives the go-ahead

to execute the next batch), then there should be no coordination overhead. However, coordination latency increases with the number of partitions (processing the next tick requires first waiting for *each* partition to forward a message to the next partition), ultimately limiting scalability.

5.2 Scalog-style time partitioning

Scalog uses a central Paxos instance to assign global slots after a scalable, partitioned set of (essentially) Paxos instances assigns local slots. The partitioned instances only mark messages as committed after receiving a global slot. This means that no message can be committed, processed, and sent back and receive a lower slot. Intuitively, this is because any commit and re-proposed message must be separated by a round of global coordination; the commit and re-proposal messages can be separately linearized to their respective points of global coordination.

The implementation is as follows:

1. A timeout is set on each partition; after the timeout, the partition sends the number of inputs that it has received to the central coordinator.
2. The coordinator waits for all partitions, then assigns a series of ticks to each partition based on the number of inputs it has received.
3. Each partition executes the ticks assigned to it on the existing batch of messages. Note that this also means that we are not limited to 1 message per batch (each partition can emulate the previous tick by executing it with whatever variable number of messages were in the previous tick).

Note that this is not entirely faithful to Scalog, since no replication is happening during coordination; during the 1st coordination, all progress is halted, and afterwards, execution is pipelined.